TEAM 8 - AAYUSH AGASHE

OPUS
SOLUTIONS

PAYMENT FAILURE INTELLIGENCE COPILOT

-OVERVIEW

GITHUB.COM/AAYUSHAGASHE/OPUS_ASSIGNMENTS/PAYFAILINTEL

EXECUTIVE SUMMARY

Payment systems experience a large volume of failures due to unclear or cryptic error codes originating from banks, payment gateways, networks, or business validation layers. These failures negatively impact customer experience, merchant revenue, and support workloads.

This project delivers a **local, offline GenAI Copilot** powered by **Retrieval-Augmented Generation (RAG)** and **Phi-2**, enabling teams to:

- Interpret payment failure reasons
- Understand root causes
- Suggest next actions
- Retrieve context from logs and documentation
- Improve support efficiency

The solution is fully local, privacy-preserving, and optimized for lightweight deployment.

DATASET SELECTED

After evaluating multiple open datasets, we selected:

Dataset: Transaction Failure Analysis Dataset
Source: GitHub – VinayakAKattimani / transaction_failure_analysis [kaggle.com]

Why This Dataset?

- Contains over 20,000 digital transactions
- Includes explicit `failure_reason`
- Contains bank names, merchant details, timestamps
- Represents realistic payment system failures
- Requires minimal augmentation for RAG

This becomes the **primary knowledge base** for the system.

TECH STACK

1. LLM: Phi-2 (Local SLM)

Lightweight and efficient for CPU-only setups

Suitable for reasoning and explanation tasks

Runs entirely offline, ensuring data privacy

2. RAG Framework: LangChain

Mature and widely adopted

Strong modularity for prompt templates, retrieval chains, evaluation tools

Better flexibility than LlamaIndex for custom pipelines

Built-in integration with FAISS

3. Embeddings Model: all-MiniLM-L3-v2

Fast and efficient on CPU

Produces high-quality embeddings for semantic similarity

384-dimensional vectors for small memory footprint

4. Vector Database: FAISS

Extremely fast similarity search

Fully local and easy to deploy

Ideal for datasets up to 100k entries

5. UI Framework: Streamlit

Quick and intuitive UI development

Enables interactive demos with minimal code

Suitable for internal POCs and presentations

6. Programming Language: Python

Mature ML and NLP ecosystem

Integrates seamlessly with LangChain, embeddings, and FAISS

SYSTEM ARCHITECTURE (TENTATIVE)

```
            Raw Dataset (GitHub)
                     |
                     ▼
       Data Cleaning & Preprocessing
                     |
                     ▼
        MiniLM Embeddings (Local CPU)
                     |
                     ▼
             FAISS Vector Store
                     |
       ┌─────────────┴─────────────┐
       ▼                           ▼
User Query (Streamlit UI)    Retriever
       |                           |
       └─────────────┬─────────────┘
                     ▼
               Phi-2 LLM
                     ▼
       Explanation + Root Cause + Resolution
```

RAG WORKFLOW

STEP 1 — DATA INGESTION

- Load dataset from GitHub
- Filter failed transactions
- Normalize fields (bank, merchant, failure_reason)

STEP 2 — CHUNKING

Transform each record into a structured text document:

```
Transaction ID: TXN123
Failure Reason: Do Not Honor
Bank: ICICI
Merchant: Amazon
Timestamp: 2024-06-12 19:52:00
```

STEP 3 — EMBEDDINGS

- Use MiniLM-L3-V2 to convert text chunks into vector embeddings

STEP 4 — VECTOR STORE (FAISS)

- Create a FAISS index
- Store vectors for fast similarity search

STEP 5 — RETRIEVAL

- User enters query (e.g., “Why do payments fail with Do Not Honor?”)
- FAISS retrieves relevant failure cases

STEP 6 — GENERATION (PHI-2)

- Phi-2 synthesizes context into explanations, root causes, and recommended actions

STEP 7 — USER INTERFACE

- Delivered through a Streamlit interface

FUNCTIONAL REQUIREMENTS

- FR1: Ingest failure logs and documentation
- FR2: Process, clean, and chunk data
- FR3: Generate embeddings
- FR4: Store vectors using FAISS
- FR5: Provide natural language query interface
- FR6: Retrieve context for a given query
- FR7: Generate explanations using Phi-2
- FR8: Provide multi-source failure context

NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Performance	Response in under 50 seconds
Scalability	Handle up to 50k failure records
Local Execution	Must run completely offline
Usability	Simple UI for non-technical users